

Retrieval-Augmented Generation (RAG)

Understanding the Complete RAG Architecture

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines a Large Language Model (LLM) with an external knowledge source. Instead of relying only on the knowledge an LLM learned during training, RAG retrieves relevant, up-to-date information from a document store at the time a question is asked, and then feeds that information to the LLM along with the question. This allows the model to generate answers that are more accurate, current, and grounded in real data, while reducing hallucination.

The Three Stages of RAG

As shown in the architecture diagram, the complete RAG pipeline is organized into three major stages: **Ingestion**, **Retrieval**, and **Generation**. Each stage plays a distinct role in turning raw data into a grounded, intelligent response.

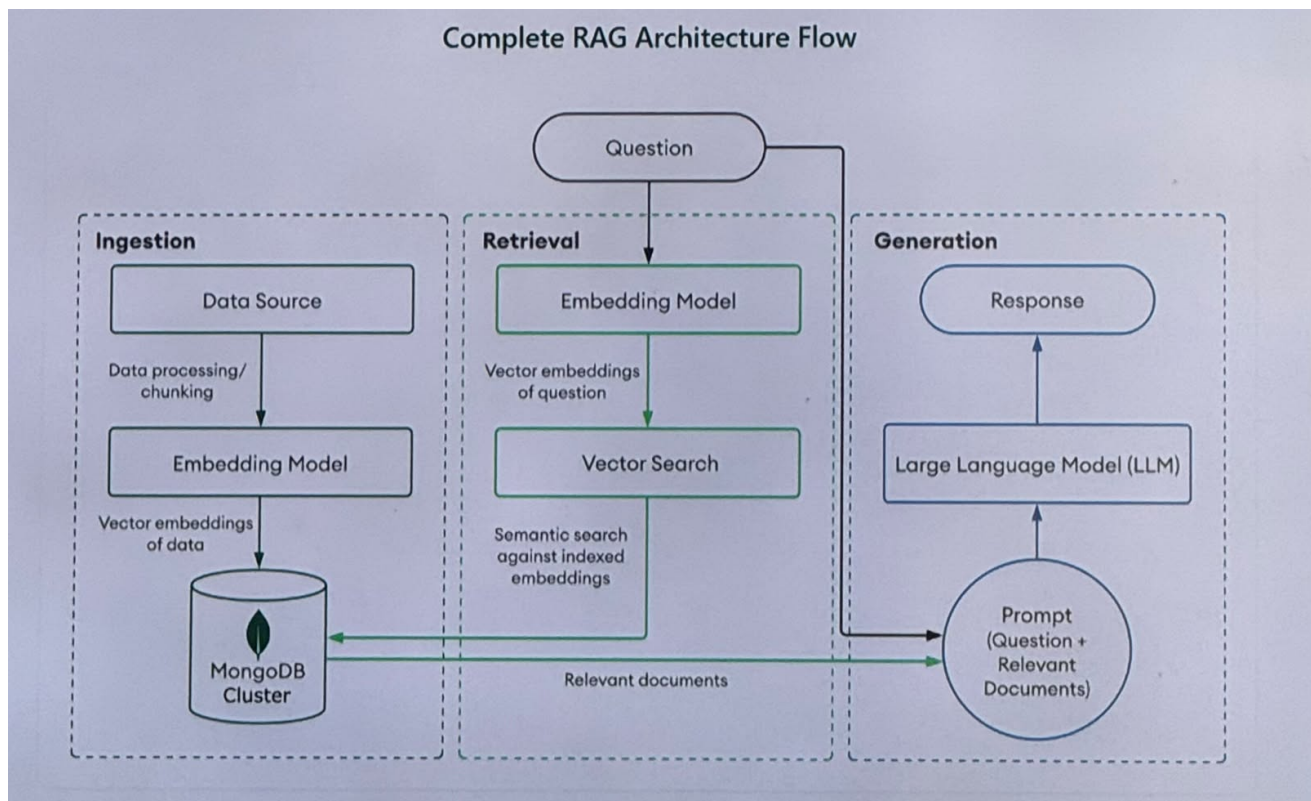


Figure 1: Complete RAG Architecture Flow

Stage 1: Ingestion

Ingestion is the offline preparation phase where raw data is converted into a searchable format and stored for later retrieval. This stage happens before any user ever asks a question.

- **Data Source** — Raw documents, files, web pages, or records that contain the knowledge you want the system to use.
- **Data processing / chunking** — The raw data is cleaned and split into smaller, manageable chunks so that each piece carries focused, retrievable meaning.
- **Embedding Model** — Each chunk of data is passed through an embedding model, which converts the text into a numerical vector representation that captures its semantic meaning.
- **Vector embeddings of data** — The resulting vectors represent the meaning of each chunk in a mathematical form that can be compared and searched efficiently.
- **MongoDB Cluster** — The vector embeddings, along with the original text, are stored in a database (here, a MongoDB cluster) that supports vector indexing and search.

Stage 2: Retrieval

Retrieval happens in real time, the moment a user submits a question. Its purpose is to find the pieces of stored data that are most relevant to that question.

- **Question** — The user's input query enters the system.
- **Embedding Model** — The same type of embedding model used during ingestion converts the question into a vector, so it can be compared with the stored data vectors.
- **Vector embeddings of question** — This is the numerical representation of the user's question.
- **Vector Search** — A semantic search is performed against the indexed embeddings in the MongoDB cluster to find the chunks of data whose meaning is closest to the question.
- **Relevant documents** — The most semantically similar chunks are retrieved from the database and passed forward to the generation stage.

Stage 3: Generation

Generation is where the system produces the final, human-readable answer using both the original question and the retrieved knowledge.

- **Prompt (Question + Relevant Documents)** — The original question is combined with the relevant documents retrieved in the previous stage to form a single, enriched prompt.
- **Large Language Model (LLM)** — This combined prompt is sent to the LLM, which now has both the question and grounding context to work with.
- **Response** — The LLM generates a final answer that is informed by the retrieved documents, making it more accurate and grounded than an answer generated from the model's internal knowledge alone.

Why RAG Matters

Without RAG, an LLM can only answer using what it learned during training, which has a fixed knowledge cutoff and no awareness of private or recently updated data. RAG solves this by dynamically fetching relevant, current information at query time and grounding the model's response in that information. This makes RAG especially valuable for use cases such as enterprise knowledge assistants, customer support bots, and systems that must answer questions about private documents or rapidly changing data.

Putting It All Together: An End-to-End Walkthrough

To see how all three stages connect, consider a simple example: a user asks a chatbot, 'What is our company's refund policy?'

- **Before the question is asked (Ingestion):** The company's policy documents have already been chunked, converted into vector embeddings, and stored in the MongoDB cluster.
- **The question arrives (Retrieval):** The question is converted into a vector using the embedding model, and a vector search is run against the MongoDB cluster to find the chunks of text most semantically similar to the question, such as the refund policy section.
- **Building the prompt (Retrieval to Generation):** The retrieved relevant documents are combined with the original question to form one enriched prompt.
- **Generating the answer (Generation):** This combined prompt is sent to the Large Language Model, which reads both the question and the retrieved policy text, and produces a final response that is accurate and grounded in the company's actual policy.

This example highlights the core advantage of RAG: the LLM never had to memorize the company's refund policy during training. Instead, the relevant facts are retrieved fresh at the moment they are needed, so the answer stays accurate even if the policy changes tomorrow.

Key Components Recap

- **Embedding Model** — Used twice: once to embed stored data during ingestion, and once to embed the incoming question during retrieval, so both can be compared in the same vector space.
- **Vector Database (MongoDB Cluster)** — Acts as the long-term memory of the system, storing both the original content and its vector representation for fast semantic lookup.
- **Vector Search** — The mechanism that finds the closest matching chunks to the question based on meaning, not just keyword matching.
- **LLM** — The reasoning engine that reads the combined prompt and produces the final natural-language response.

Prompt Engineering vs Fine-Tuning vs RAG

All three approaches aim to improve how a language model responds, but they differ significantly in method, cost, and the type of problem they solve best. The table below summarizes the key differences.

Aspect	Prompt Engineering	Fine-Tuning	RAG
Definition	Carefully crafting the input instructions/prompt given to a pretrained model to guide its behavior, without changing the model itself.	Further training a pretrained model on a specific, labeled dataset to adjust its internal weights for a particular task or domain.	Combining a pretrained model with an external retrieval system that fetches relevant data at query time and feeds it into the prompt.
Modifies model weights?	No	Yes	No
Knowledge source	Only what the model already knows, shaped by how the prompt is phrased.	Knowledge baked into the model during the fine-tuning training process.	External, up-to-date knowledge base searched live at the time of the query.
Handles new/changing data?	No — limited to the model's existing knowledge.	No, not automatically — requires retraining whenever data changes.	Yes — simply update the document store; no retraining needed.
Cost & effort	Low — just rewriting text instructions; no training required.	High — needs labeled data, compute resources, and training time.	Moderate — requires setting up embeddings, a vector database, and a retrieval pipeline.
Risk of hallucination	Can still be high, since the model relies purely on its own internal knowledge.	Reduced for the trained domain, but can still hallucinate outside it.	Lower — responses are grounded in retrieved, real documents.
Best suited for	Quick adjustments in tone, format, or task framing for general use cases.	Teaching the model a new skill, style, or domain-specific behavior in depth.	Question-answering and assistants that need current, factual, or private data.
Update frequency	Instant — just change the prompt text.	Slow — requires a new training run for each update.	Fast — just add or update documents in the knowledge base.

Summary

Prompt engineering is the fastest and cheapest way to steer model behavior but is limited by the model's existing knowledge. Fine-tuning goes deeper by retraining the model itself, making it powerful for specialized tasks but costly and slow to update. RAG strikes a balance: it keeps the base model unchanged while connecting it to a live, external knowledge source, making it the preferred approach for keeping AI responses accurate, current, and grounded in real data.